Лабораторная работа 3. Функциональное тестирование

Имя студента: Цароев Альберт Казебкович
Группа: ПИЖ-б-о-23-1(1)
Предмет: Тестирование и отладка программного обеспечения
Претендую на оценку: Отлично

---

1. Описание метода и стратегии тестирования

1.1. Виды функционального тестирования

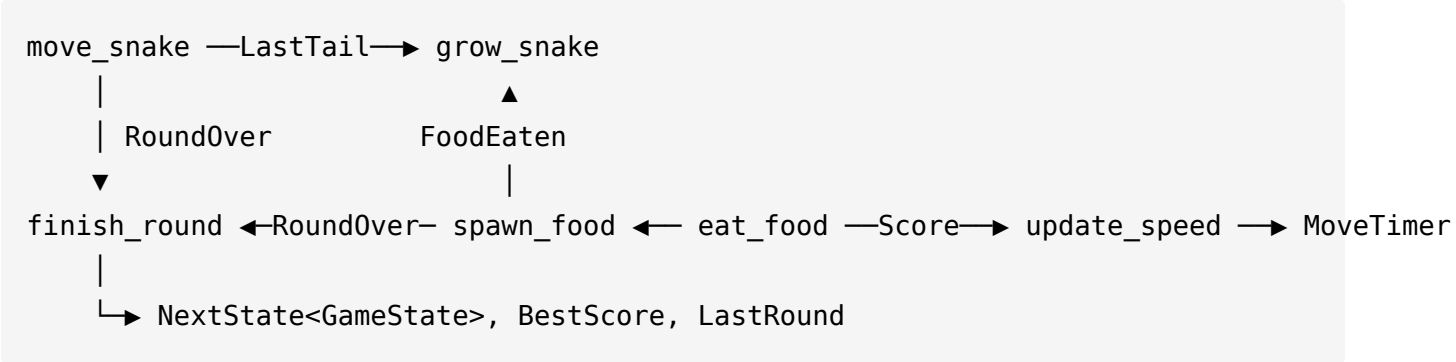
Вид	Что проверяет	Где в проекте
Компонентное (модульное)	отдельную функцию или систему в изоляции	ЛР5
Интеграционное	взаимодействие компонентов	**эта работа**
Системное	всю систему на соответствие требованиям	сценарии ЛР2 (игра целиком)
Приёмочное	готовность к передаче заказчику	ручной чек-лист ЛР2
Регрессионное	что изменения не сломали старое	`cargo test` после каждого изменения
Smoke	критичные функции сразу после сборки	`cargo test --test lb2 -- mt1` (игра стартует)

Выбрано интеграционное тестирование. В ECS поведение распределено по системам, которые не вызывают
друг друга напрямую. Они общаются через данные: одна система пишет компонент, ресурс или сообщение,

другая читает. Каждая система может быть верна по отдельности, а ошибка возникнет на стыке, например
из-за неправильного порядка или из-за отложенной команды, которая ещё не применилась.

1.2. Стыки, которые проверяются

Цепочка одного хода (порядок задан `SnakeSet` и `.chain()` в src/game/mod.rs):



Подход - **восходящая интеграция**: функции `core` проверены отдельно (ЛР1, ЛР5), здесь системы
собраны в настоящее расписание `SnakeLogicPlugin` и проверяются по цепочкам.

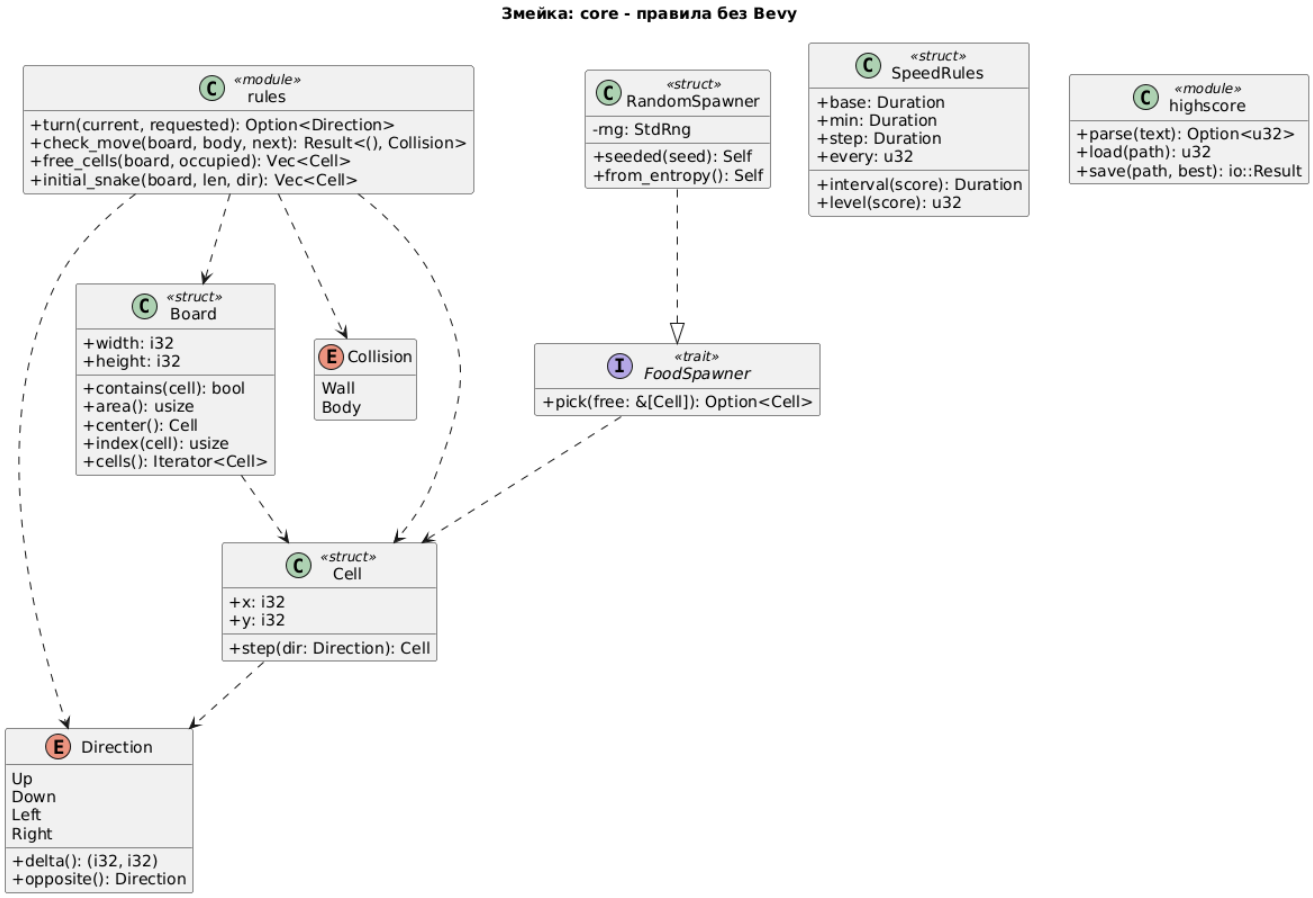
1.3. Процесс функционального тестирования

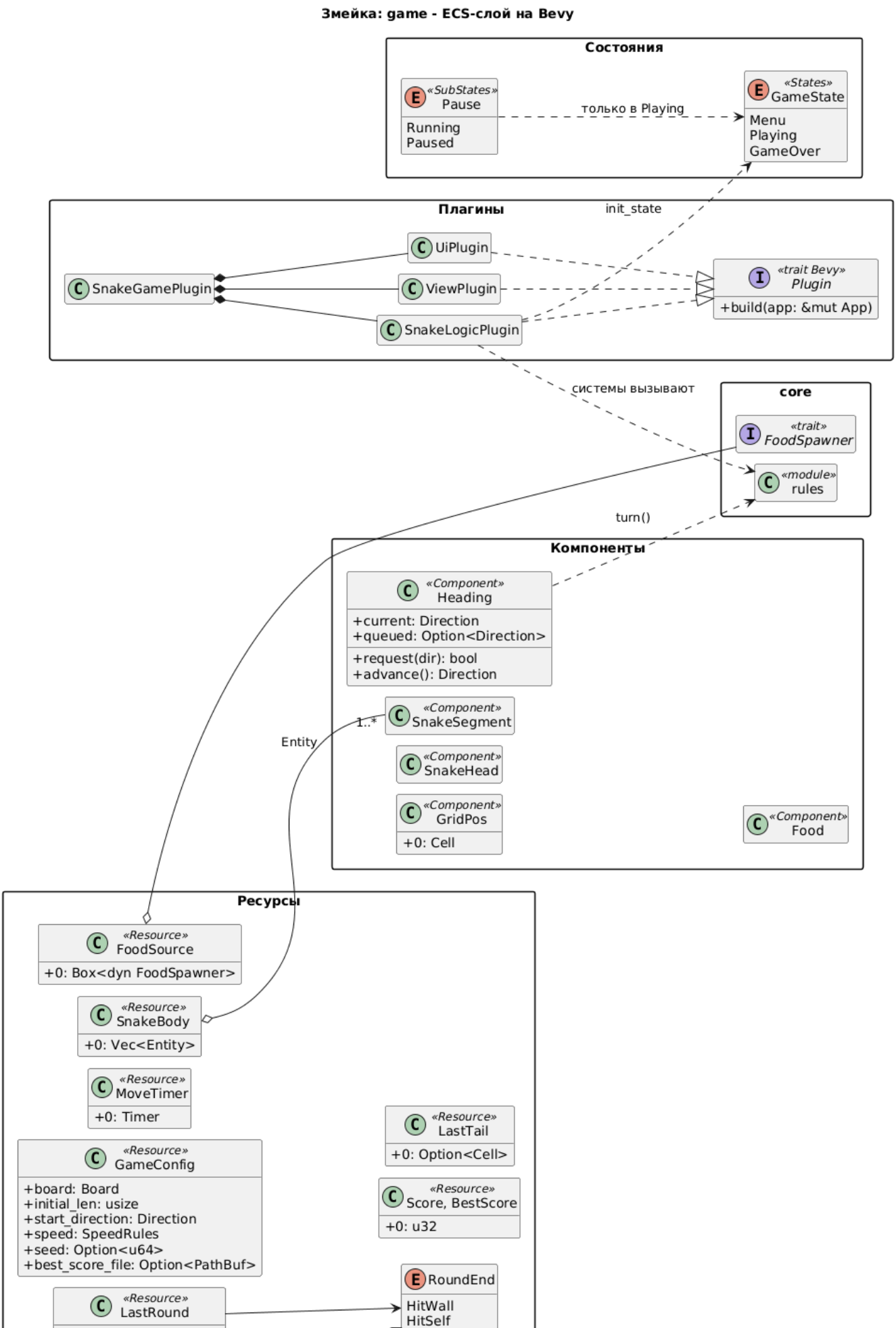
1. **Анализ требований.** Требования к правилам игры сведены в таблицу FR (раздел 3.1).
2. **Тестовые сценарии.** На каждое требование - сценарий, проходящий через нужный стык систем.
3. **Тестовые данные.** Конфигурация поля и список клеток еды (`ScriptedSpawner`). С ними исход каждого сценария предсказуем заранее.
4. **Выполнение и сравнение.** `cargo test --test lb3` : фактическое состояние ECS сравнивается с ожидаемым в `assert` .
5. **Дефекты.** Журнал в разделе 4.2.
6. **Повторное тестирование** после исправления: полный прогон `cargo test` , то есть регрессия всех работ.

1.4. Запуск

```
cargo test --test lb3
scripts/test.sh lb3 --report
```

2. Диаграмма классов




Змейка: game - ECS-слой на Bevy
Состояния
«SubStates»
Pause
Running
Paused
только в Playing
«States»
GameState
Menu
Playing
GameOver
Плагины
init_state
UiPlugin
SnakeGamePlugin
ViewPlugin
SnakeLogicPlugin
«trait Bevy»
Plugin
+build(app: &mut App)
системы вызывают
core
«trait»
FoodSpawner
«module»
rules
turn()
Компоненты
«Component»
Heading
+current: Direction
+queued: Option<Direction>
+request(dir): bool
+advance(): Direction
1..*
Entity
«Component»
SnakeSegment
«Component»
SnakeHead
«Component»
GridPos
+0: Cell
«Component»
Food
Ресурсы
«Resource»
FoodSource
+0: Box<dyn FoodSpawner>
«Resource»
SnakeBody
+0: Vec<Entity>
«Resource»
MoveTimer
+0: Timer
«Resource»
LastTail
+0: Option<Cell>
«Resource»
GameConfig
+board: Board
+initial_len: usize
+start_direction: Direction
+speed: SpeedRules
+seed: Option<u64>
+best_score_file: Option<PathBuf>
«Resource»
Score, BestScore
+0: u32
RoundEnd
HitWall
HitSelf
«Resource»
LastRound

3. Тестовые материалы

Код: integration.rs.

3.1. Требования и трассировка

№	Требование	Стык систем	Тест
FR-1	Enter в меню начинает партию	`menu_keys` → `NextState` → `start_round` → `spawn_food`	ЛР2 `mt1`
FR-2	Змейка делает 1 ход за интервал	`advance_timer` → `move_due` → `move_snake`	ЛР2 `mt2`
FR-3	Разворот на 180° запрещён	`steer` → `Heading` → `move_snake`	ЛР2 `mt4`
FR-4	Еда удлиняет змейку на 1 сегмент в клетке бывшего хвоста	`move_snake` (LastTail) → `eat_food` (FoodEaten) → `grow_snake`	`fr4_new_segment_appears_where_tail_was`
FR-5	Каждые 5 очков интервал меньше на 10 мс	`eat_food` (Score) → `update_speed` (MoveTimer)	`fr5_speed_increases_every_five_points`
FR-6	Укус себя завершает партию	`move_snake` (RoundOver) → `finish_round`	`fr6_biting_self_ends_round`

№	Требование	Стык систем	Тест
		→ GameOver	
FR-7	Заполнение всего поля - победа	`grow_snake` → `spawn_food` (RoundOver BoardFilled) → `finish_round`	`fr7_filling_board_wins`
FR-8	Рекорд сохраняется, счёт обнуляется в новой партии	`finish_round` (BestScore) → `start_round` (Score)	`fr8_best_score_survives_restart`
FR-9	Еда не появляется на змейке	`grow_snake` (новый сегмент через Commands) → `spawn_food`	`food_never_spawns_on_snake`

3.2. Тестовые данные

Тест	Поле	Длина	Еда	Действия
FR-4	20×15	3	(10,7)	1 ход
FR-5	20×15	3	(10,7)…(14,7)	4 хода, затем ещё 1
FR-6	20×15	5	(0,0)	↑, ←, ↓ по ходу на каждое
FR-7	3×1	2	единственная свободная (2,0)	1 ход
FR-8	20×15	3	(10,7), (11,7)	11 ходов до стены, Enter

Тест	Поле	Длина	Еда	Действия
FR-9	20×15	3	(10,7), затем (7,7) - клетка, куда встанет новый сегмент	1 ход

FR-9 проверяет конкретный риск. `grow_snake` создаёт сегмент через `Commands`, то есть отложенно. Если бы
Bevy не применил команды до `spawn_food`, тот не увидел бы новый сегмент и положил бы еду на змейку.
Сценарий специально просит положить еду именно туда.

4. Результаты выполнения тестов

4.1. Протокол

Все 6 тестов пройдены. Протокол: results.txt.

```
Протокол: lb3
Система:  Linux 7.0.0-38-generic x86_64
Rust:     rustc 1.98.1 (48a229cea 2026-09-01)
Команда:  cargo test --test lb3

test integration::food_never_spawns_on_snake ... ok
test integration::fr4_new_segment_appears_where_tail_was ... ok
test integration::fr5_speed_increases_every_five_points ... ok
test integration::fr6_biting_self_ends_round ... ok
test integration::fr7_filling_board_wins ... ok
test integration::fr8_best_score_survives_restart ... ok

test result: ok. 6 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out
```

4.2. Журнал дефектов

Дефекты найдены при первом запуске собранной игры (smoke). Оба - на стыке `UiPlugin` и машины состояний.

№	Симптом	Причина	Исправление
D-1	Паника при запуске: `Resource does not exist`	Кнопки читают `NextState<Pause>`, но `Pause` - подсостояние `Playing`, и в меню этого ресурса нет	Параметр стал `Option<ResMut<NextState<Pause>>>`
D-2	Паника при запуске: `Resource does not exist` (после D-1)	Шрифты грузились в `PreStartup`, а `OnEnter(Menu)` начального состояния выполняется **раньше** `PreStartup`	Шрифты загружаются при сборке плагина (`FromWorld`)

Дальнейшая работа: регрессионный тест для D-1/D-2 - приложение с `UiPlugin` без окна проходит все экраны.